

实验报告

实验项目名称 实验八 视图、索引、序列和同义词的创建和使用

实 验 室 云计算研究中心实验室 520

所属课程名称 数 据 库

实 验 日 期 2025.12.3

班 级 24 大数据 2 班

学 号 倪伟丰

姓 名 2024110089

【实验环境】

- Oracle11g 及以上

【实验目的】

- 掌握视图和索引的创建及使用方法
- 掌握序列和同义词的创建及使用方法
- 掌握用 Oracle 数据字典查看用户创建的视图、索引、序列和同义词的方法

【实验结果提交方式】

- 实验 8-3
自行完成本补充实验，记录实验结果。
- 将本实验报告存放在“XXXXXXXXX-8.docx”文件中，其中“XXXXXXXXX”是学号，并在教师规定的时间内通过 BB 系统提交该文件。

【实验 8-3 的实验结果】

自行完成补充实验，并记录实验结果。

1. 创建一个名为“EmpIncome”的视图，其中包含所有雇员的编号、姓名和工资，其中，工资=薪金（SAL）+奖金（COMM）；

```
CREATE VIEW EmpIncome AS
SELECT EMPNO, ENAME, (SAL + COMM) AS INCOME
FROM EMP;

SELECT * FROM EMPINCOME;
```

	EMPNO	ENAME	INCOME
1	7369	SMITH	800
2	7499	ALLEN	1900
3	7521	WARD	1750
4	7566	JONES	2975
5	7654	MARTIN	2650
6	7698	BLAKE	2850
7	7782	CLARK	2450
8	7788	SCOTT	3000
9	7839	KING	5000
10	7844	TURNER	1500
11	7876	ADAMS	1100
12	7900	JAMES	950
13	7902	FORD	3000
14	7934	MILLER	1300

2. 创建一个名为“Employee10_30”的视图，其中包含部门“10”和“30”的所有雇员的编号、部门编号、姓名、雇佣日期、工作和薪金等信息；

```
CREATE VIEW Employee10_30 AS
SELECT EMPNO, ENAME, HIREDATE, JOB, SAL
FROM EMP
WHERE DEPTNO IN (10, 30);
```

	EMPNO	DEPTNO	ENAME	HIREDATE	JOB	SAL
1	7499	30	ALLEN	20/02/1981 12:00:00	SALESMAN	1600
2	7521	30	WARD	22/02/1981 12:00:00	SALESMAN	1250
3	7654	30	MARTIN	28/09/1981 12:00:00	SALESMAN	1250
4	7698	30	BLAKE	01/05/1981 12:00:00	MANAGER	2850
5	7782	10	CLARK	09/06/1981 12:00:00	MANAGER	2450
6	7839	10	KING	17/11/1981 12:00:00	PRESIDENT	5000
7	7844	30	TURNER	08/09/1981 12:00:00	SALESMAN	1500
8	7900	30	JAMES	03/12/1981 12:00:00	CLERK	950
9	7934	10	MILLER	23/01/1982 12:00:00	CLERK	1300

3. 利用视图 “Employee10_30”，将部门 30 的所有销售员的薪金增加 25% ；

	EMPNO	DEPTNO	ENAME	HIREDATE	JOB	SAL
1	7499	30	ALLEN	20/02/1981 12:00:00	SALESMAN	2000
2	7521	30	WARD	22/02/1981 12:00:00	SALESMAN	1562.5
3	7654	30	MARTIN	28/09/1981 12:00:00	SALESMAN	1562.5
4	7698	30	BLAKE	01/05/1981 12:00:00	MANAGER	2850
5	7782	10	CLARK	09/06/1981 12:00:00	MANAGER	2450
6	7839	10	KING	17/11/1981 12:00:00	PRESIDENT	5000
7	7844	30	TURNER	08/09/1981 12:00:00	SALESMAN	1875
8	7900	30	JAMES	03/12/1981 12:00:00	CLERK	950
9	7934	10	MILLER	23/01/1982 12:00:00	CLERK	1300

4. 创建一个名为 “EmpMgr” 的视图，其中包含雇员的编号、姓名、部门名称和上级姓名的视图；

```
CREATE VIEW EmpMgr AS
SELECT E1.EMPNO, E1.ENAME, DNAME, E2.ENAME AS MGRNAME
from EMP E1,EMP E2, DEPT
where E1.MGR=E2.EMPNO(+) and E1.DEPTNO=DEPT.DEPTNO;

SELECT * FROM EmpMgr;
```

	EMPNO	ENAME	DNAME	MGRNAME
1	7902	FORD	RESEARCH	JONES
2	7788	SCOTT	RESEARCH	JONES
3	7900	JAMES	SALES	BLAKE
4	7844	TURNER	SALES	BLAKE
5	7654	MARTIN	SALES	BLAKE
6	7521	WARD	SALES	BLAKE
7	7499	ALLEN	SALES	BLAKE
8	7934	MILLER	ACCOUNTING	CLARK
9	7876	ADAMS	RESEARCH	SCOTT
10	7782	CLARK	ACCOUNTING	KING
11	7698	BLAKE	SALES	KING
12	7566	JONES	RESEARCH	KING
13	7369	SMITH	RESEARCH	FORD
14	7839	KING	ACCOUNTING	(null)

5. 创建一个名为“deptSumSal”的视图，其中包含各部门的名称和雇员总薪金；

```
CREATE VIEW deptSumSal AS
select DNAME, SUM(SAL) AS totalSal
FROM EMP LEFT JOIN DEPT ON EMP.DEPTNO = DEPT.DEPTNO
GROUP BY DNAME;

SELECT * FROM deptSumSal;
```

	DNAME	TOTALSAL
1	ACCOUNTING	8750
2	RESEARCH	10875
3	SALES	10800

6. 创建一个名为“JobSal”的视图，其中包含公司提供的各种工作的名称、平均薪金、最高薪金和最低薪金：

```
CREATE VIEW JobSAI AS

select JOB, AVG(SAL) AS AVG_SAL, MAX(SAL) AS MAX_SAL, MIN(SAL) AS MIN_SAL

from EMP

GROUP BY JOB;

SELECT * FROM JobSAI;
```

[illegible]

7. 创建一个名为“SeqDeptNo”的序列，该序列的起始值是 50，步长为 10；

```
CREATE SEQUENCE SeqDeptNo
```

```
START WITH 50
```

```
INCREMENT BY 10;
```

```
Sequence SEQDEPTNO created.
```

8. 在 DEPT 表中插入 2 个新部门，见下表，其中部门编号使用序列 “SeqDeptNo” 生成；

新部门信息

DEPTNO	DNAME	LOC
50	PURCHASING	BOSTON
60	PERSONNEL	NEW YORK

```
INSERT INTO DEPT (DEPTNO,DNAME,LOC) VALUES
(SeqDeptNo.NEXTVAL,'PURCHASING','BOSTON');
```

```
INSERT INTO DEPT (DEPTNO,DNAME,LOC) VALUES
(SeqDeptNo.NEXTVAL,'PERSONNEL','NEW YORK');
```

```
SELECT * FROM DEPT;
```

```
COMMIT;
```

	DEPTNO	DNAME	LOC
1	10	ACCOUNTING	NEW YORK
2	20	RESEARCH	DALLAS
3	30	SALES	CHICAGO
4	40	OPERATIONS	BOSTON
5	50	PURCHASING	BOSTON
6	60	PERSONNEL	NEW YORK

9. 创建一个名为 “部门” 的同义词，该同义词代表的是 DEPT 表；

```
CREATE SYNONYM 部门 FOR DEPT;
```

```
select * from 部门;
```

	DEPTNO	DNAME	LOC
1	10	ACCOUNTING	NEW YORK
2	20	RESEARCH	DALLAS
3	30	SALES	CHICAGO
4	40	OPERATIONS	BOSTON
5	50	PURCHASING	BOSTON
6	60	PERSONNEL	NEW YORK

10. 利用同义词 “部门”，查询部门 10 和 50 的部门名称；

```
select DEPTNO, DNAME from 部门 where deptno in (10, 50);
```


	INDEX_NAME	INDEX_TYPE	TABLE_NAME	UNIQUENESS
1	PK_EMP	NORMAL	EMP	UNIQUE
2	IDXENAME	NORMAL	EMP	NONUNIQUE
3	IDXDNOsal	NORMAL	EMP	NONUNIQUE
4	PK_DEPT	NORMAL	DEPT	UNIQUE

15. 删除上面创建的 Employee10_30 视图和所有同义词、序列和索引。

```
drop view DeptSumSal;
```

```
drop view EmplIncome;
```

```
drop view EmpMgr;
```

```
drop view Employee10_30;
```

```
drop view JobSal;
```

```
View DEPTSUMSAL dropped.
```

```
View EMPINCOME dropped.
```

```
View EMPMGR dropped.
```

```
View EMPLOYEE10_30 dropped.
```

```
View JOBSAL dropped.
```

```
drop synonym 部门;
```

```
Synonym 部门 dropped.
```

```
drop SEQUENCE SeqDeptNo;
```

```
Sequence SEQDEPTNO dropped.
```

```
drop index idxEname;
```

```
drop index idxDnoSal;
```

```
Index IDXENAME dropped.
```

```
Index IDXDNOsal dropped.
```

【实验思考】

1. 既然企业业务活动中产生的数据是存放在基本表中的,为什么还要创建视图呢?
2. 简述只读视图的创建方法及其作用。
3. 举例说明用带有“WITH CHECK OPTION”子句的 CREATE VIEW 语句创建检查视图的作用。
4. 什么样的视图是不能更新的?
5. 如何利用 Oracle 系统的数据字典, 查看用户所拥有的视图的相关信息?
6. 简述创建惟一性索引的作用。
7. 如何利用 Oracle 系统的数据字典, 查看用户所拥有的索引的相关信息?
8. 简述序列和同义词的作用。

【思考结果】

将思考结果记录在本实验报告中。

1. 既然企业业务活动中产生的数据是存放在基本表中的,为什么还要创建视图呢?
视图可以将复杂的 SQL 查询(如多表连接、聚合函数、子查询等)封装起来。用户只需要像查询普通表一样查询视图即可;
对于经常使用的查询逻辑, 定义为视图可以避免重复编写相同的 SQL 代码, 提高开发效率。
视图可以限制用户访问表中的特定行或列。通过只授予用户访问视图的权限而不是底层表的权限, 可以隐藏敏感数据。例如例题中的 Employee10_30 限制只能访问部门 10 与 30 的员工;
如果底层基本表的结构发生变化, 可以通过修改视图的定义来保持对外接口不变, 从而避免修改上层应用程序的代码。

2. 简述只读视图的创建方法及其作用。

CREATE VIEW 视图名称 AS

SELECT ...

WITH READ ONLY;

作用是: 防止数据被修改, 强制该视图只能用于查询 (SELECT), 禁止通过该视图执行任何 DML 操作。

3. 举例说明用带有“WITH CHECK OPTION”子句的 CREATE VIEW 语句创建检查视图的作用。

WITH CHECK OPTION 的作用是强制通过视图进行的修改 (INSERT 或 UPDATE) 必须满足视图定义中的 WHERE 条件。如果修改后的数据不再满足视图的筛选条件, 数据库将拒绝该操作。

例如:

CREATE OR REPLACE VIEW Employee10_30 AS

SELECT * FROM emp

WHERE deptno IN (10, 30)

WITH CHECK OPTION;

此时不允许插入 deptno 不为 10 或 30 的员工。

4. 什么样的视图是不能更新的?

显式只读视图: 定义时包含了 WITH READ ONLY 子句的视图。

包含聚合函数或分组的视图: 使用了聚合函数 (如 SUM, AVG, MAX 等)。

包含集合操作符的视图: 使用了 UNION, INTERSECT, MINUS 等集合操作符。

包含去重操作的视图：使用了 **DISTINCT** 关键字。

包含表达式的视图：对于由表达式计算出来的列（例如 `sal * 12`），虽然视图本身可能允许更新其他列，但计算列本身不可更新。

5. 如何利用 **Oracle** 系统的数据字典，查看用户所拥有的视图的相关信息？

使用以下 **SQL** 语句可以查看当前用户拥有的所有视图的名称、定义语句以及是否只读等信息：

```
SELECT VIEW_NAME, TEXT, READ_ONLY FROM USER_VIEWS;
```

6. 简述创建惟一性索引的作用。

创建惟一性索引（**Unique Index**）主要有以下两个作用：

保证数据的唯一性：

它强制索引列中的值不能重复（除了 **NULL** 值），从而在物理存储层面确保了业务数据的唯一性（例如：确保“身份证号”或“员工工号”不重复）。

提高查询效率：

它能极大加快对该列的精确查找速度。因为数据库引擎知道该列的值是唯一的，一旦找到匹配的记录，就会立即停止搜索，无需扫描剩余的数据。

7. 如何利用 **Oracle** 系统的数据字典，查看用户所拥有的索引的相关信息？

使用以下 **SQL** 语句可以查看当前用户拥有的所有索引的名称、类型、所属表以及唯一性状态：

```
select INDEX_NAME, INDEX_TYPE, TABLE_NAME, UNIQUENESS from USER_INDEXES;
```

8. 简述序列和同义词的作用。

序列（**Sequence**）的作用

- 自动生成唯一数值：序列是一个数据库对象，用于生成一系列唯一的整数。
- 主键生成器：最常见的用途是为表的主键列提供自动增长的值。
- 高并发支持：它独立于表存在，可以被多个用户共享，能够高效地在多用户并发环境下生成不重复的序号。

同义词（**Synonym**）的作用

- 简化引用：为复杂的数据库对象名称（如用户名.表名）创建一个简短、易记的别名，方便编写 **SQL**。
- 位置透明性：如果底层对象的名称或位置（所属用户）发生变化，只需修改同义词定义，而无需修改应用程序代码。
- 安全性：可以隐藏对象的实际名称和所有者，用户只需知道同义词即可操作，无需知道底层表的具体细节。

实验成绩：_____ 批阅老师：_____ 批阅日期：_____